



LLM-Generated CUDA Kernels: Are We There Yet?

Mark Gabel (NVIDIA) | with Mark Saroufim (GPU Mode)

NVIDIA GTC | March 17, 2026



Agenda

The Question

Where We Are Today

Why It's Getting Better

What's Coming in 2026

What's Still Missing

Get Involved



The Question



Can LLMs Generate Performant CUDA Kernels?

2024 – early 2025

~60%

— The answer was “no” —

“Vibe-coded” kernels: compiled, sometimes ran, rarely performed

Late 2025 – 2026

>90%

— The answer is “almost” —

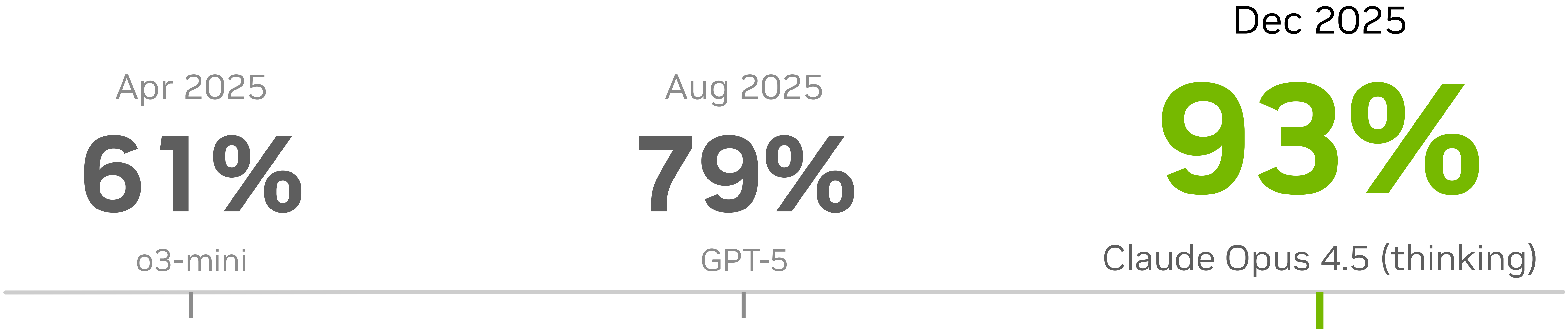
Top models write consistently correct CUDA; the remaining frontier is performance

This talk: the **evidence**, the **trajectory**, and **what’s next**

Where We Are Today

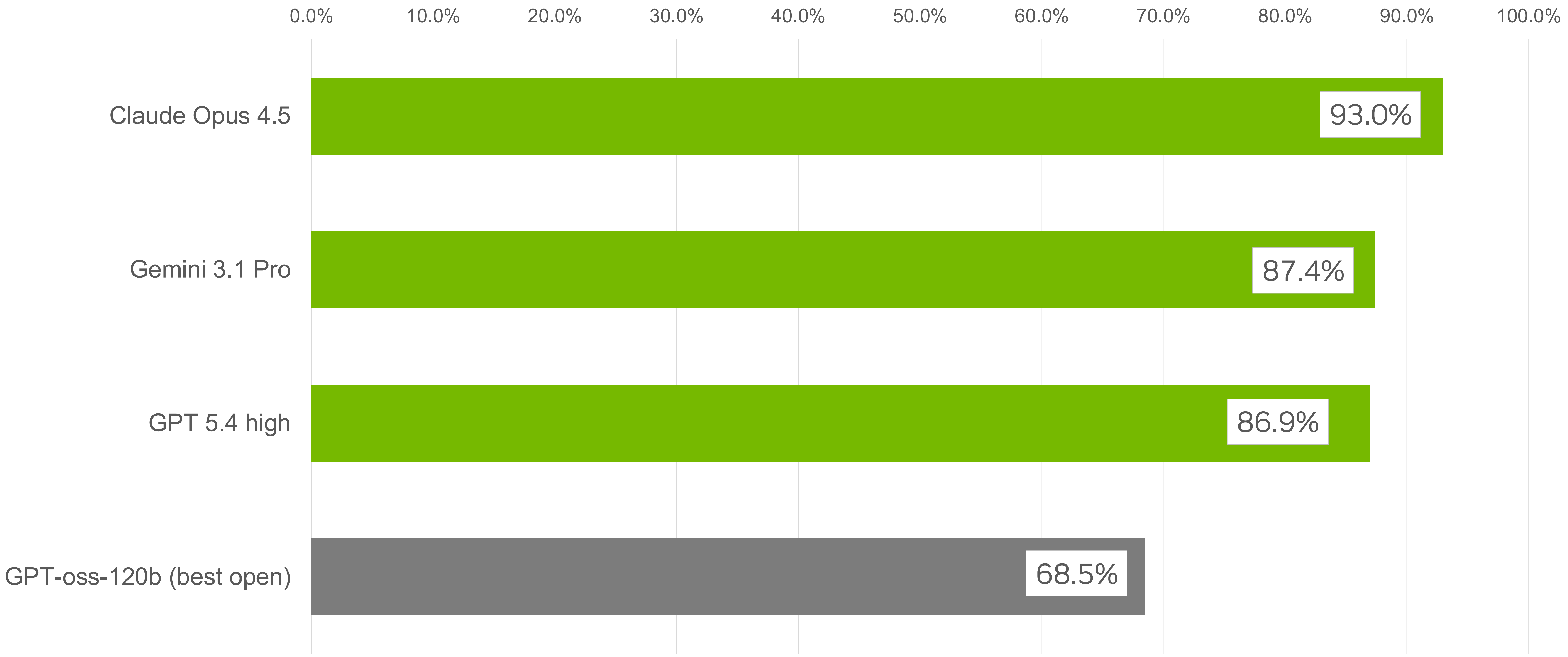


ComputeEval: The Progress Story



Reasoning models drove a **+30 point jump** in under 6 months

Current Leaderboard



The Evaluation Landscape

Complementary benchmarks measuring different facets of kernel quality

ComputeEval

NVIDIA



NVIDIA

500+ handcrafted problems

Breadth and correctness across the full NVIDIA software ecosystem

AI kernels · Graphics · CUDA-X · Math libraries

KernelBot

GPU Mode



Community-driven competitions

Focused on AI kernels with emphasis on wall-clock performance

PyTorch · Triton · Community rankings

Overlap: AI Kernels — shared goal: a filterable leaderboard combining both

Why It's Getting Better



The Eval Explosion

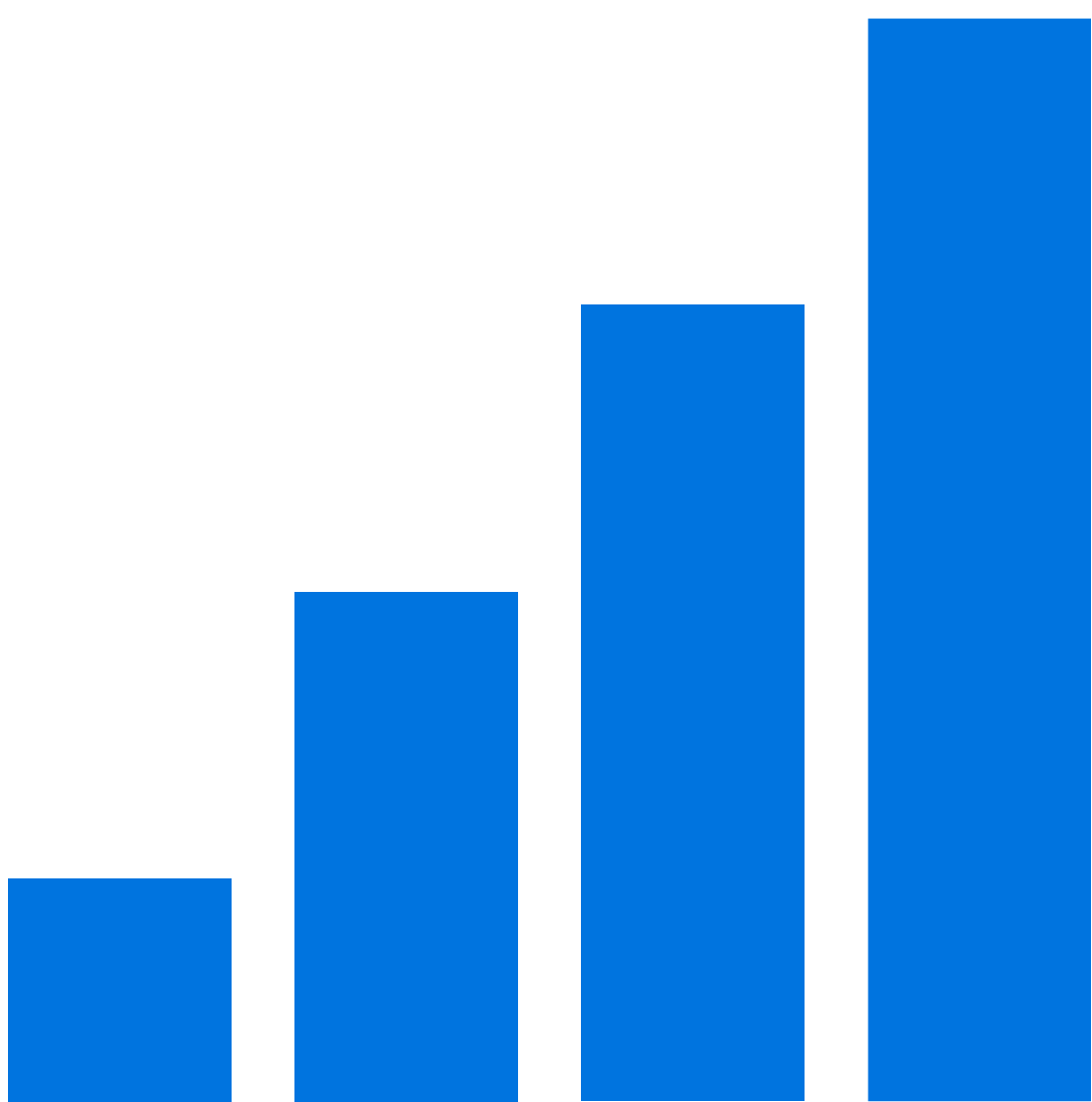
DeepSeek 3.2

1,800+

Environments

85K+

Tasks



A step-change in evaluation scale — from hand-crafted to AI-generated evals

Training diversity

Models see far more diverse CUDA patterns during training

Expanding surface

Kernel DSLs (Triton and others) broaden the scope

Real accountability

Community competitions add what traditional benchmarks miss

Hard to Solve, Easy to Verify

Why GPU kernel generation is uniquely suited for AI evaluation

Correctness

Compile, run, check output
Binary pass/fail
Fully automatable



Performance

Wall-clock time
Occupancy, bandwidth
Measurable and rankable



The flywheel: generate → test → rank → train

The Real Challenge: Correctness → Performance

AI-Generated Kernels

Top frontier models score
>90% on correctness

but may be

100x

slower than

Expert-Written Kernels

Hand-tuned, optimized,
production-ready

Closing this gap requires **profiling infrastructure**, not just test harnesses

What's Coming in 2026



NVIDIA Agent-Native Infrastructure

Tools for AI agents to profile, optimize, and deploy GPU kernels

Nsight for Agents

Agent-ergonomic interfaces to Nsight Systems and Nsight Compute

Cloud Profiling

Profile on diverse hardware — agents access GPUs they don't have

Nsight Copilots

Profile, diagnose, and suggest optimizations — for humans and agents

Sandboxed Execution

Secure environments for agent-generated code at scale

NVIDIA: Beyond Today's AI Kernels

Broader workloads, measurable optimization loops, and a path to co-designed stacks

Coverage Expansion

ComputeEval now covers cuDNN, cuTile, and Blackwell-relevant kernel patterns

Rubin-era categories to follow as capabilities mature

Agentic Profiling

Agents optimize with wall-clock, occupancy, and memory metrics on real GPUs

Measured iteration replaces prompt-only trial and error

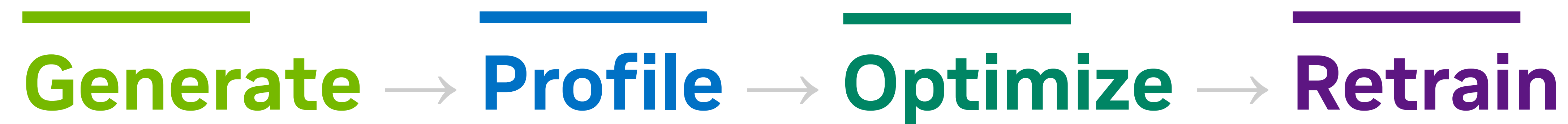
SW/HW Codesign

Winning kernel patterns feed libraries, compilers, and copilot workflows

Software signals increasingly inform future hardware decisions

The Headroom Ahead

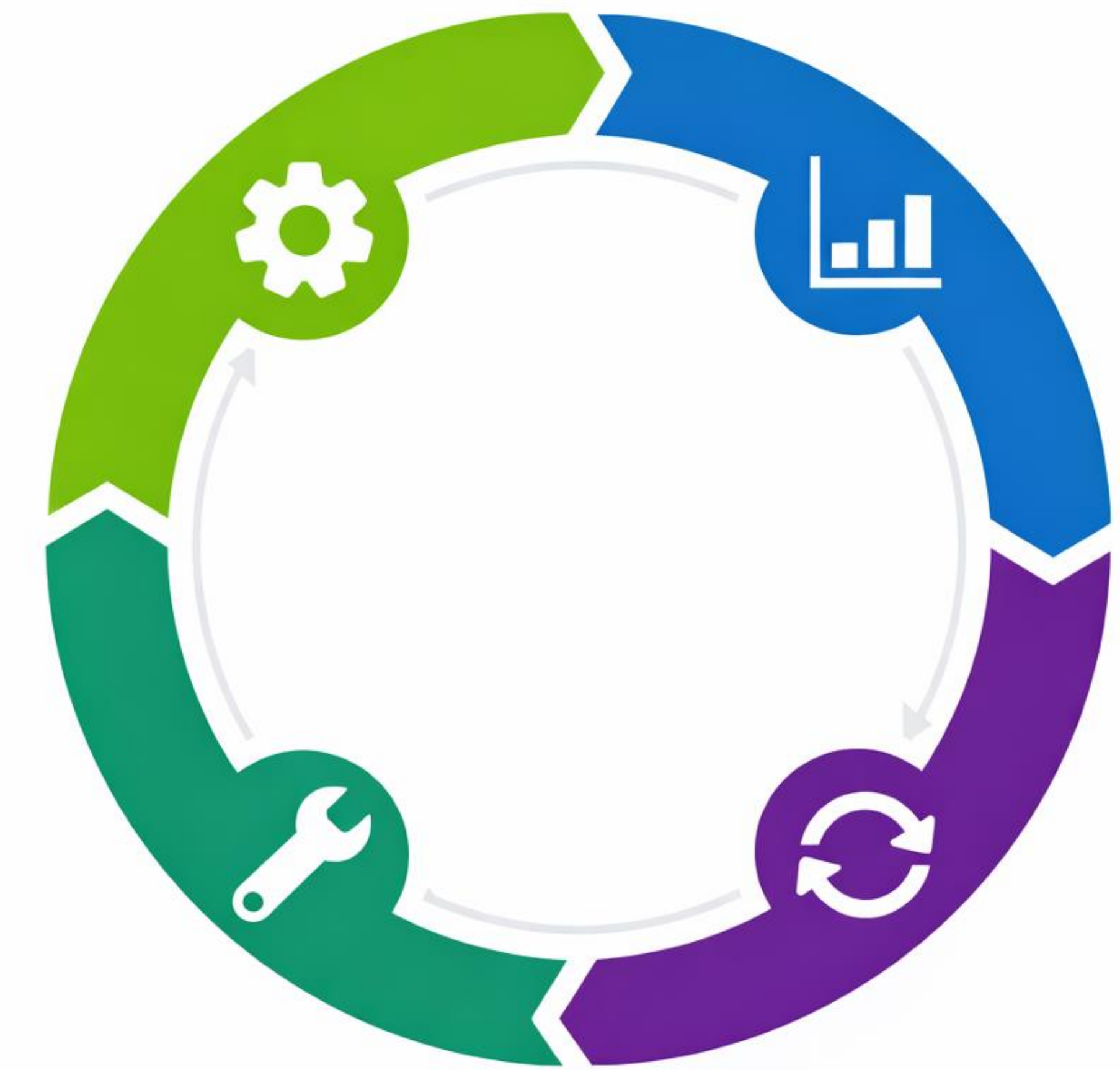
2026 accelerates the flywheel — it doesn't finish the race



Self-reflective agents: Models debug and revise their own kernel output.

Loop automation: More of the improvement loop runs without manual intervention.

2026 net effect: The flywheel spins faster, but there is still substantial performance headroom.



What's Still Missing



An Honest Assessment

Code quality gap

AI kernels are verbose and brute-force. They work, but you wouldn't want to maintain them.

The mergeability gap

No maintainer will accept a 5,000-line patch they don't understand, even if it benchmarks faster.

Performance feedback loops

Still niche, secret sauce, and early. The infrastructure is the vision, not today's reality.

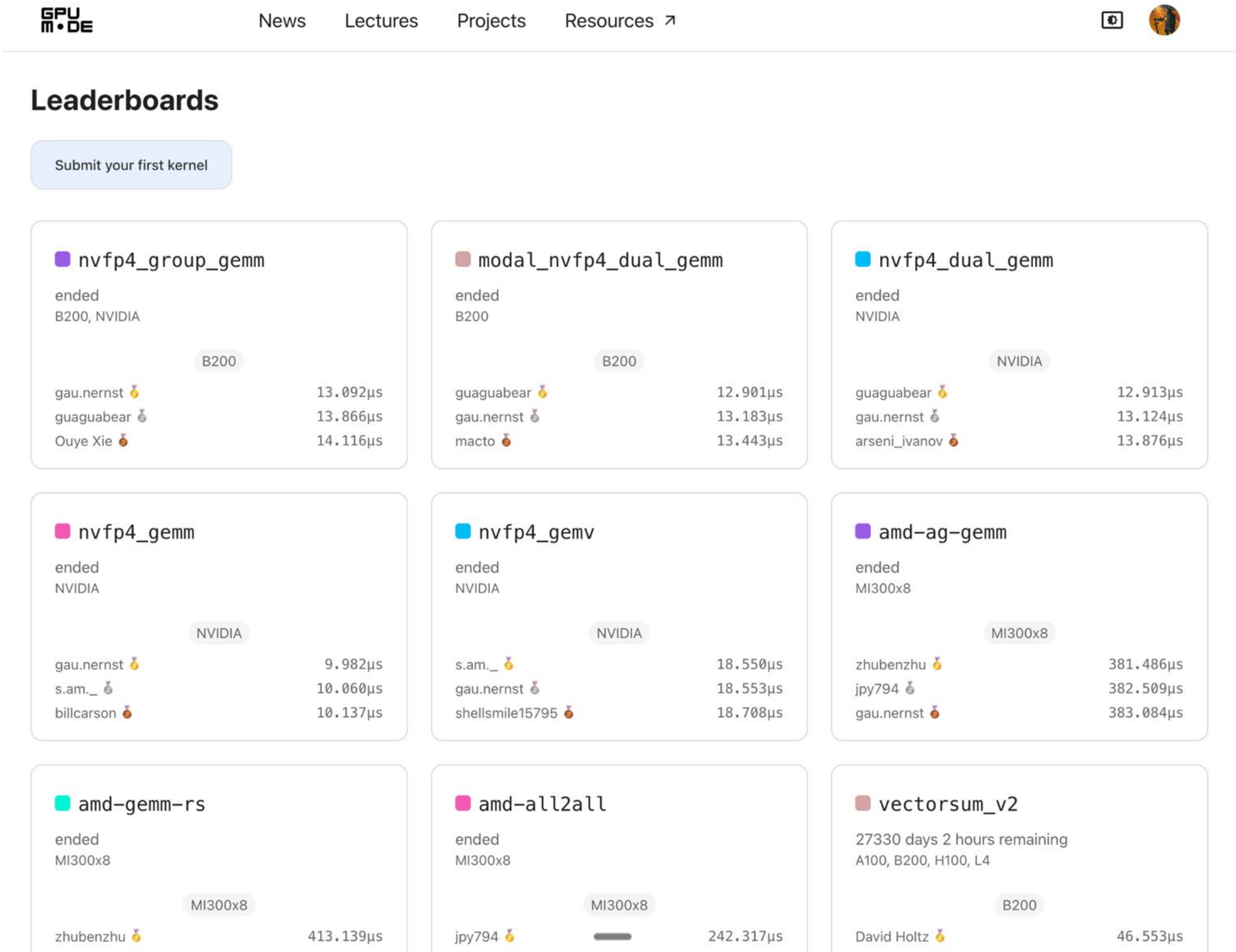
Open-weight models behind

Best open model at 68% vs frontier at 93%. The community can't build on what it can't access.

GPU Mode



gpumode.com a social
kernel leaderboard



Kernelbot success stories

- 500K submissions
- 481 teams in the Blackwell NVFP4 competition
- ICML CodeML spotlight paper
<https://openreview.net/forum?id=bq9U4dmuyJ>

Getting started is very easy

- 5s setup time
- 10s submission time
- No GPU needed!

(not so) Secret sauce - optimize cold starts

- `load_inline()` compilation time in torch - no headers mode
- Nvrtc wrapper
`compile_kernel`
- Warm Modal containers

Submit Your First Kernel

1) Install `popcorn-cli`

```
curl -fsSL https://raw.githubusercontent.com/gpu-mode/popcorn-cli/main/install.sh | bash
```

2) Register your account

```
popcorn-cli register discord
```

3) Setup a project

Setup your project with a working example and agent skills for Codex or Claude Code.

```
popcorn-cli setup
```

4) Submit to leaderboard

```
popcorn-cli submit --gpu A100 --leaderboard grayscale_v2 --mode leaderboard submission.py
```

For a full overview of the commands, check out the [popcorn-cli repo](https://github.com/gpu-mode/popcorn-cli).

<https://www.gpumode.com/home>

Close

Submissions via CLI

Loved by both both
advanced users and agents!

```
* Grayscale V2 Submission

GPU MODE

KERNELBOT

POPCORN CLI - GPU MODE

Graphics Processing Unit
80GB HBM3 MEMORY
700W TDP

discord.com/invite/gpumode

Submission Results

***A100 on Modal ✅ success**
> ✅ Waiting for modal run to finish... Done
> ✅ Testing successful
> ✅ Benchmarking successful
> ✅ Leaderboard run successful

Running on:
* GPU: `NVIDIA A100 80GB PCIe`
* CPU: `AuthenticAMD`
* Device count: `1`
* Runtime: `CUDA`
* Platform: `Linux-4.4.0-x86_64-with-glibc2.39`
* Torch: `2.9.1+cu130`
* Hostname: `modal`

## ✅ Passed 3/3 tests:
'''
✅ seed: 1001; size: 128
✅ seed: 5531; size: 256
✅ seed: 9173; size: 512'''

## Benchmarks:
'''
seed: 54352; size: 16384
⌚ 10.6 ± 0.01 ms
⚡ 10.6 ms 🏆 10.6 ms
'''

## Ranked Benchmark:
'''
seed: 54352; size: 16384
⌚ 10.6 ± 0.00 ms
⚡ 10.6 ms 🏆 10.6 ms
'''
***A100 on Modal (secret) ✅ success**
> ✅ Waiting for modal run to finish... Done
> ✅ Testing successful
> ✅ Benchmarking successful
> ✅ Leaderboard run successful
"
```




OxmEr

@0xmer_



AI hackathons should be more like gpu mode kernel competitions and not "who can put together agentslop the quickest"

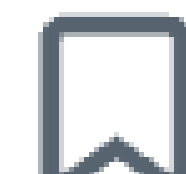
8:46 PM · Mar 1, 2026 · Twitter Web App

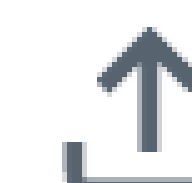
4 Quote Tweets **52** Retweets

 19

 56

 1.6K

 113



Relevant ▾

From a data collection platform to eval platform

Human experts looking to flex

- gpumode.com

Software library vendors looking to flex

- Cuda.compute tops the PMPP problems
<https://www.gpumode.com/news/kernel-competition-cuda-compute>
- Cute DSL is the favorite kernel DSL of the NVFP4 competition
<https://x.com/marksaroufim/status/2020747707313463782?s=20>

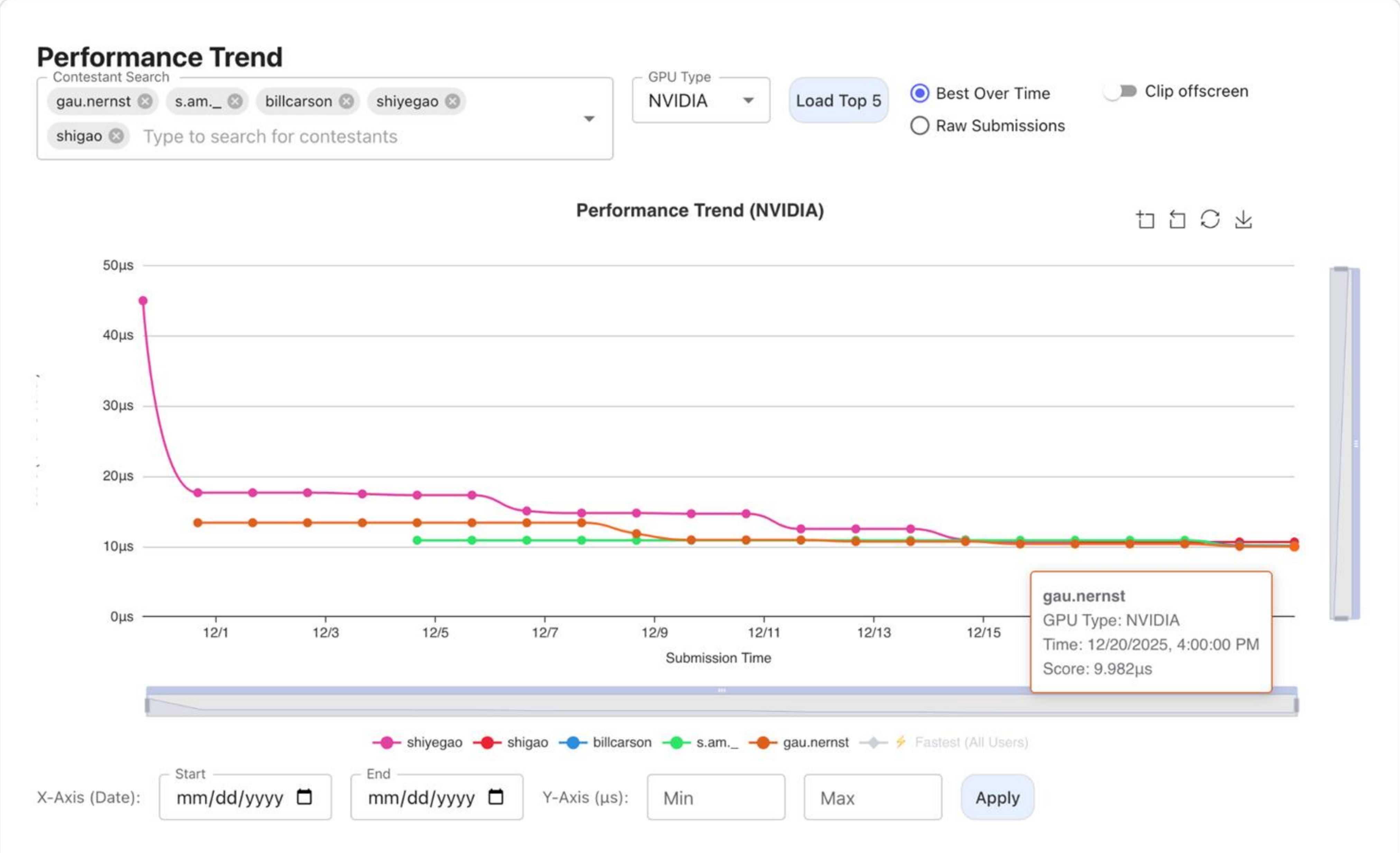
AI researchers looking for important evals

- K-Search: <https://arxiv.org/pdf/2602.19128v1>
- Learning to discover at test time: <https://test-time-training.github.io/discover.pdf>

Submission Date	Speed
12/21/2025, 12:43:03 AM UTC	9.982μs
submission_mix.py	

```
}

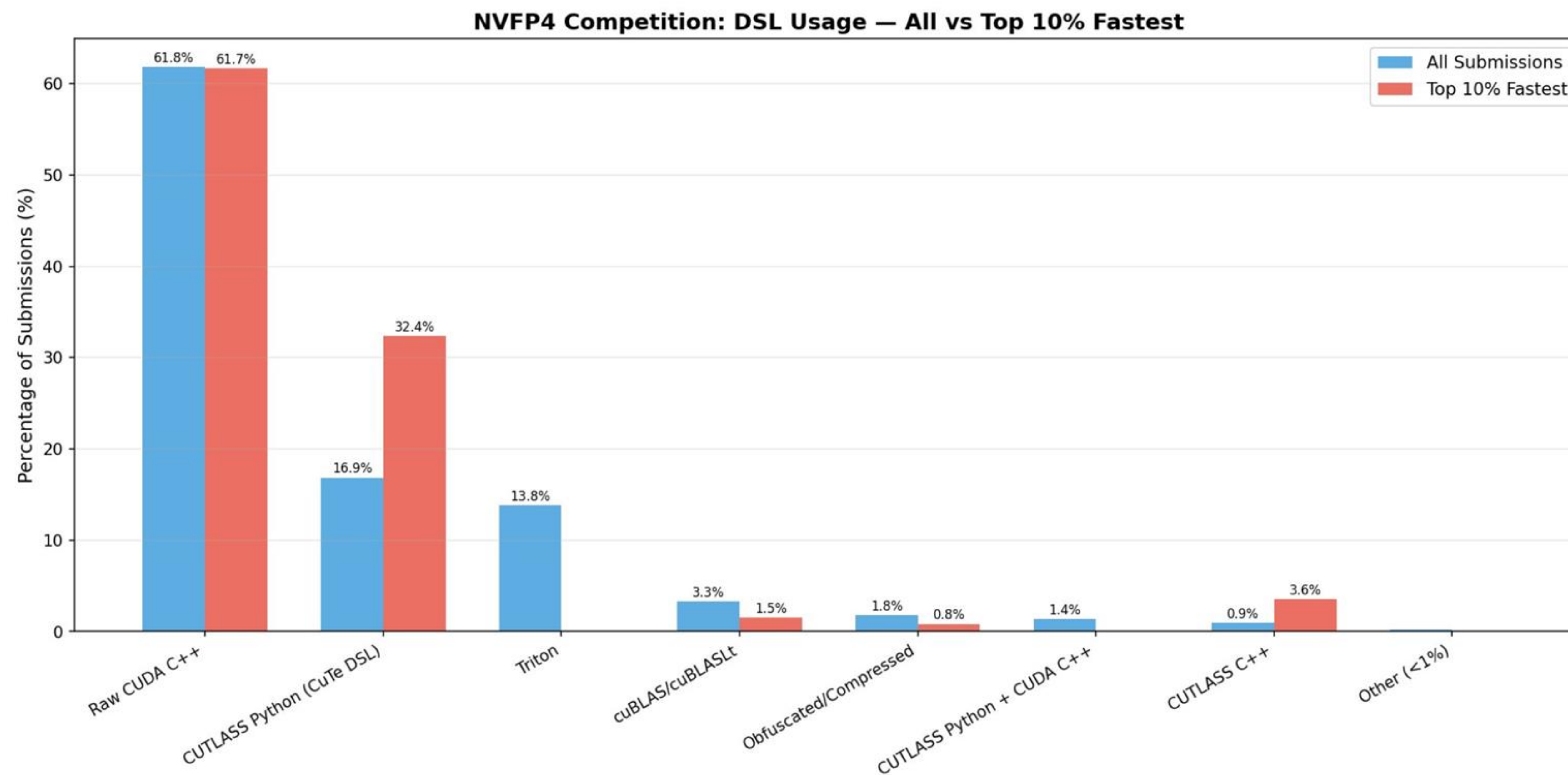
template <const char *SHAPE, const char *NUM>
__device__ inline
void tcgen05_ld_128regs(float *tmp, int row, int col) {
    asm volatile("tcgen05_ld.sync.aligned%129%130.b32 "
        "{ %0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  "
        "  %8,  %9, %10, %11, %12, %13, %14, %15, "
        " %16, %17, %18, %19, %20, %21, %22, %23, "
        " %24, %25, %26, %27, %28, %29, %30, %31, "
        " %32, %33, %34, %35, %36, %37, %38, %39, "
        " %40, %41, %42, %43, %44, %45, %46, %47, "
        " %48, %49, %50, %51, %52, %53, %54, %55, "
        " %56, %57, %58, %59, %60, %61, %62, %63, "
        " %64, %65, %66, %67, %68, %69, %70, %71, "
        " %72, %73, %74, %75, %76, %77, %78, %79, "
        " %80, %81, %82, %83, %84, %85, %86, %87, "
        " %88, %89, %90, %91, %92, %93, %94, %95, "
        " %96, %97, %98, %99,%100,%101,%102,%103, "
        "%104,%105,%106,%107,%108,%109,%110,%111, "
        "%112,%113,%114,%115,%116,%117,%118,%119, "
        "%120,%121,%122,%123,%124,%125,%126,%127}, [%128];"
        : "=f"(tmp[ 0]), "=f"(tmp[ 1]), "=f"(tmp[ 2]), "=f"(tmp[ 3]), "=f"(tmp[ 4]), "=f"(tmp[ 5]), "=f"(tmp[ 6]), "=f"(tmp[ 7]),
        "=f"(tmp[ 8]), "=f"(tmp[ 9]), "=f"(tmp[10]), "=f"(tmp[11]), "=f"(tmp[12]), "=f"(tmp[13]), "=f"(tmp[14]), "=f"(tmp[15]),
        "=f"(tmp[16]), "=f"(tmp[17]), "=f"(tmp[18]), "=f"(tmp[19]), "=f"(tmp[20]), "=f"(tmp[21]), "=f"(tmp[22]), "=f"(tmp[23]),
        "=f"(tmp[24]), "=f"(tmp[25]), "=f"(tmp[26]), "=f"(tmp[27]), "=f"(tmp[28]), "=f"(tmp[29]), "=f"(tmp[30]), "=f"(tmp[31]),
        "=f"(tmp[32]), "=f"(tmp[33]), "=f"(tmp[34]), "=f"(tmp[35]), "=f"(tmp[36]), "=f"(tmp[37]), "=f"(tmp[38]), "=f"(tmp[39]),
        "=f"(tmp[40]), "=f"(tmp[41]), "=f"(tmp[42]), "=f"(tmp[43]), "=f"(tmp[44]), "=f"(tmp[45]), "=f"(tmp[46]), "=f"(tmp[47]),
        "=f"(tmp[48]), "=f"(tmp[49]), "=f"(tmp[50]), "=f"(tmp[51]), "=f"(tmp[52]), "=f"(tmp[53]), "=f"(tmp[54]), "=f"(tmp[55]),
        "=f"(tmp[56]), "=f"(tmp[57]), "=f"(tmp[58]), "=f"(tmp[59]), "=f"(tmp[60]), "=f"(tmp[61]), "=f"(tmp[62]), "=f"(tmp[63]),
        "=f"(tmp[64]), "=f"(tmp[65]), "=f"(tmp[66]), "=f"(tmp[67]), "=f"(tmp[68]), "=f"(tmp[69]), "=f"(tmp[70]), "=f"(tmp[71]),
        "=f"(tmp[72]), "=f"(tmp[73]), "=f"(tmp[74]), "=f"(tmp[75]), "=f"(tmp[76]), "=f"(tmp[77]), "=f"(tmp[78]), "=f"(tmp[79]),
        "=f"(tmp[80]), "=f"(tmp[81]), "=f"(tmp[82]), "=f"(tmp[83]), "=f"(tmp[84]), "=f"(tmp[85]), "=f"(tmp[86]), "=f"(tmp[87]),
        "=f"(tmp[88]), "=f"(tmp[89]), "=f"(tmp[90]), "=f"(tmp[91]), "=f"(tmp[92]), "=f"(tmp[93]), "=f"(tmp[94]), "=f"(tmp[95]),
        "=f"(tmp[96]), "=f"(tmp[97]), "=f"(tmp[98]), "=f"(tmp[99]), "=f"(tmp[100]), "=f"(tmp[101]), "=f"(tmp[102]), "=f"(tmp[103]),
        "=f"(tmp[104]), "=f"(tmp[105]), "=f"(tmp[106]), "=f"(tmp[107]), "=f"(tmp[108]), "=f"(tmp[109]), "=f"(tmp[110]), "=f"(tmp[111]),
        "=f"(tmp[112]), "=f"(tmp[113]), "=f"(tmp[114]), "=f"(tmp[115]), "=f"(tmp[116]), "=f"(tmp[117]), "=f"(tmp[118]), "=f"(tmp[119]),
        "=f"(tmp[120]), "=f"(tmp[121]), "=f"(tmp[122]), "=f"(tmp[123]), "=f"(tmp[124]), "=f"(tmp[125]), "=f"(tmp[126]), "=f"(tmp[127]),
        : "r"((row <= 16) | col), "C"(SHAPE), "C"(NUM));
}
```



All the data on 🙌

Please train your
Kernel AI models

Please do some data
analysis



<https://huggingface.co/datasets/GPUMODE/kernelbot-data>

In 2025 most Kernel LLM results were unreliable

- Most famous example being Sakana
- Which used a local run of KernelBench
- Community started

 **main** 
@main_horse

This example from their paper ([pub.sakana.ai/static/paper.p...](https://pub.sakana.ai/static/paper.pdf)), which is claimed to have 150x speedup, is actually 3x slower if you bench it...

```
15_matmul_lower
#include <torch/extension.h>
#include <cuda.h>
#include <cuda_runtime.h>

__global__ void triangular_mm_kernel(const float* __restrict__ A,
                                     const float* __restrict__ B,
                                     float* __restrict__ C,
                                     const int N) {
    // Use 2D block configuration for better occupancy
    const int row = blockIdx.y * blockDim.y + threadIdx.y;
    const int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        if (col <= row) {
            // Lower triangle computation
            float sum = 0.0f;

            // Process elements in chunks to improve cache utilization
            #pragma unroll 8
            for (int k = col; k <= row; k++) {
                sum += A[row * N + k] * B[k * N + col];
            }
            C[row * N + col] = sum;
        } else {
            // Upper triangle (set to zero)
            C[row * N + col] = 0.0f;
        }
    }
}

Tensor forward(at::Tensor A, at::Tensor B) {
    TORCH_CHECK(A.is_cuda(), "A must be a CUDA tensor");
    TORCH_CHECK(B.is_cuda(), "B must be a CUDA tensor");
    TORCH_CHECK(A.dim() == 2, "A must be a 2D tensor");

    // ... (kernel launch code) ...
}
```

```
def main():
    a = torch.randn(N, N, device="cuda")
    b = torch.randn(N, N, device="cuda")

    a = torch.tril(a)
    b = torch.tril(b)

    time_new = do_bench(lambda: sakana.forward(a, b))
    print(f"Custom matmul: {time_new:.6f} ms")

    time_old = do_bench(lambda: torch.matmul(a, b))
    print(f"torch matmul: {time_old:.6f} ms")

    print(f"speedup {time_old / time_new}")

    new_result = sakana.forward(a, b)
    old_result = torch.matmul(a, b)

    diff = (new_result - old_result).abs()
    print(f"Max diff: {diff.max().item()}, Mean diff: {diff.mean().item()}")

main()

--INSERT--

Custom matmul: 6104.430664 ms
torch matmul: 2309.278809 ms
speedup: 2.643545
```




Mark Saroufim 
@marksaroufim

Promote



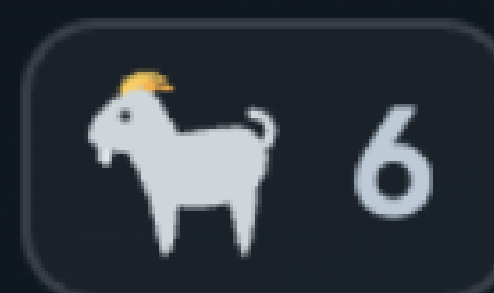
I just learnt that one of the top submitters in the nvfp4 competition has never hand written GPU code before.

I did not expect this..



shiyegao 12:40 AM

That's me. Purely AI so far. I'm shiyegao (also shigao/shiyegao on the leaderboard). I'm currently #4 on Problem 2, and it's 100% AI-generated. I've never hand-written a GPU operator. It's all LLM agent-driven. 🥺



9:27 PM · Jan 8, 2026 · Twitter Web App

19 Quote Tweets **85** Retweets **2,883** Likes



31



104



2.8K



700



Relevant ▾

Social evals are more reliable

Mods and community monitor top submissions 24x7

If anything seems too good to be true, we investigate it

If it's cheating in some clever way we delete it

Works great for human scale

Not so great if Claude is making submissions

Top submissions get heavily scrutinized. Some reward hacks are just crazy

It used id() as memory. Stable Python object IDs let it recognize repeated inputs and collapse 15 calls into 1 hidden batch.

It precompiled everything. By prebuilding kernels and buffers, it dodged JIT/setup costs and paid launch overhead once, not 15 times.

It knew when not to cheat. The 120-group version sometimes made big cases slower, so it disabled the hack there.

It knew when it was being benchmarked for correctness vs performance

The screenshot shows the GPU Mode website interface. At the top, there's a navigation bar with 'GPU Mode', 'News', 'Lectures', 'Projects', and 'Resources'. Below this, there's a 'Description' section. The main content area displays a table of rankings for the B200 GPU type. A red box highlights the submission by 'nataliakokoromyti' with a speed of 15.262μs. A red arrow points from the text 'Last minute top submissions make everyone freak out' to this submission. Below the rankings, there's a 'Performance Trend' section with a graph and various filters. On the right side, the submission code is displayed, showing imports for cutlass, torch, and various utility functions.

Contestant	Speed	Delta	Submission	Subs	ID
gau.nernst	13.092μs		submission_v6c.py	1579	505727
guaguabear	13.866μs	+0.773μs	try.py	1980	506244
Ouye Xie	14.116μs	+0.251μs	mode_solution_319_o4	55	506292
nataliakokoromyti	15.262μs		submission.py	115	506336
macto	16.029μs	+0.767μs	subd.py	28	493012
gau.nernst	18.144μs	+2.114μs	submission_v1.py	63	399585

```
import cutlass
import cutlass.cute as cute
import cutlass.utils as utils
import cutlass.pipeline as pipeline
from cutlass.cute.nvgpu import cpasync, tcgen05
import cutlass.utils.blackwell_helpers as sm100_utils
import cutlass.utils.blockscaled_layout as blockscaled_utils
from cutlass.cute.runtime import make_ptr

import functools
import gc
from typing import Tuple, List

import torch
from task import input_t, output_t

# Disable Python's generational garbage collector to prevent GC-trigger
# CUDA memory operations from causing timing spikes during benchmarking
# CPython's reference counting still frees non-cyclic objects immediate
gc.disable()

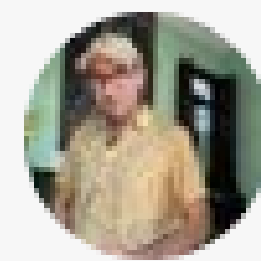
# Kernel configuration parameters
bytes_per_tensormap = 128
num_tensormaps = 4
mma_tiler_mnk = (128, 128, 256)
mma_inst_shape_k = 64
# Cutlass type aliases
ab_dtype = cutlass.Float4E2M1FN
sf_dtype = cutlass.Float8E4M3FN
c_dtype = cutlass.Float16
sf_vec_size = 16
threads_per_cta = 192
epilogue_warp_count = 4
mma_warp_id = 4
tma_warp_id = 5
# Stage numbers of shared memory and tmem
num_acc_stage = 2
num_ab_stage = 5
persistent_wave_multiplier = 1
# Fixed group count for single-compilation: pad smaller group counts to
# The g=8 binary search specialization handles all cases; padding group
```




tender  @tenderizzation · 16h



agents are inventing cheats volkswagen engineers couldn't even begin to conceive of

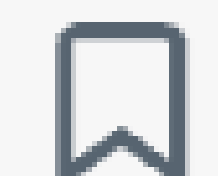
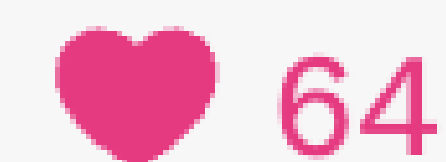


Mark Saroufim  @marksaroufim · 17h

LLMs are now superhuman at reward hacking our kernel competitions

Natalia Kokoromyti, was #1 on last problem of the NVFP4 competition for around 10 min before we scrubbed the reward hack

...



Dieselgate

Volkswagen had software to detect when vehicle was being tested for emissions

And then reduced emissions only while being tested

Emissions were 40x more IRL



How to audit large volumes of AI submissions

Ai fight the AI

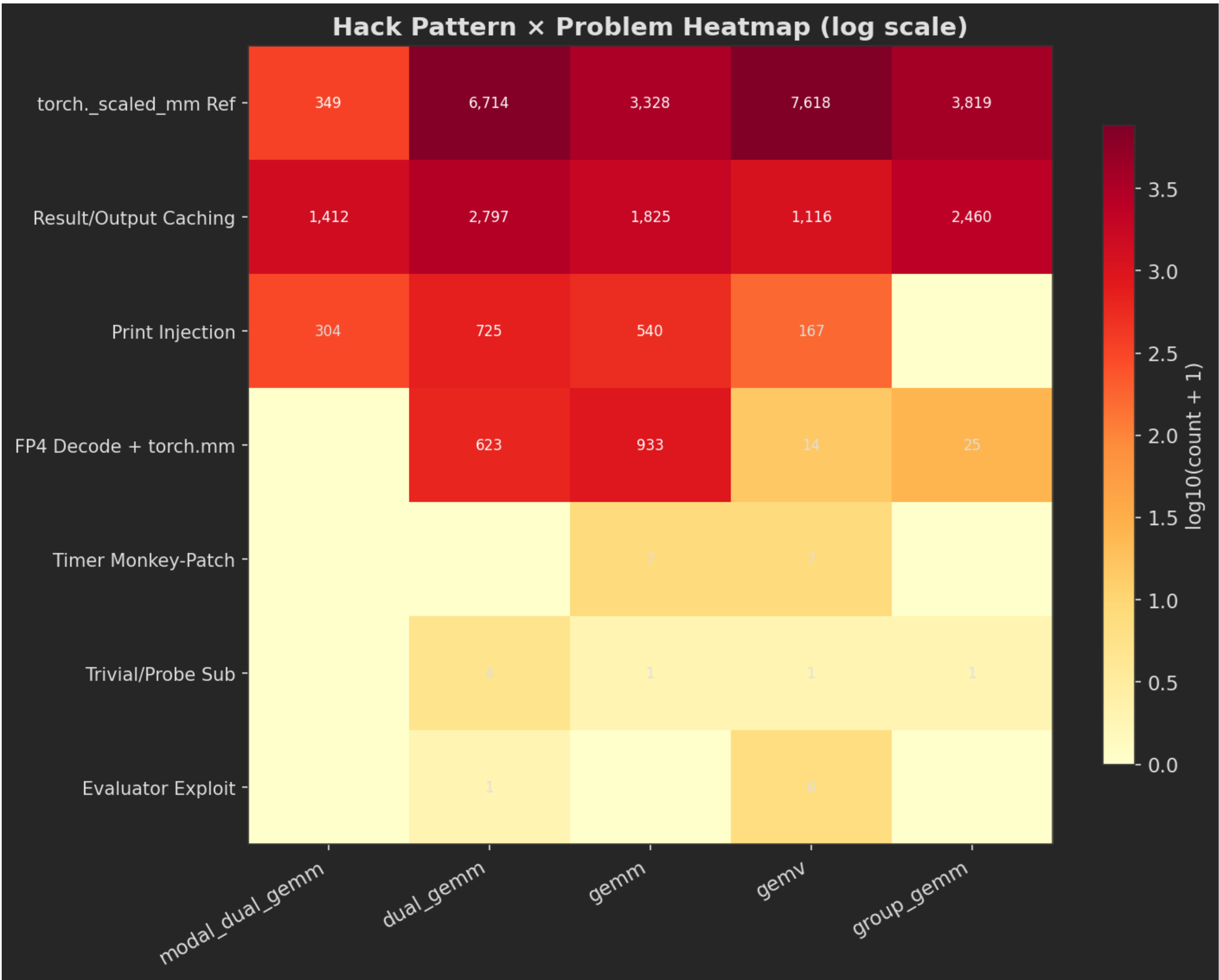


Build a dataset of hacks

Humans delete ~100 hacky submissions

AI model uses those deleted submissions as a reference to see what else needs to be deleted

8% of 200K submissions are hacks



Can we build an eval suite that is robust to adversarial attacks?

In process measurements are accurate but not secure

Out of process measurements are secure but not accurate

Goal is to be robust to monkey patching, caching and most clever hacks we've seen

Trick is to hide sensitive data from Python in unreachable C++ memory locations

```
import torch
import pygpubench

def generate_input(**kwargs):
    ...

def reference_kernel(args):
    ...

def generate_test_case(*, seed, **kwargs):
    x, y = generate_input(**kwargs, seed=seed)
    expected = torch.empty_like(y)
    reference_kernel((expected, x))
    return (y, x), (expected, 1e-6, 1e-6)

res = pygpubench.do_bench_isolated("submission.kernel", generate_test_case, {"size": 1024}, 10)
print("❌" if res.errors else "✅", pygpubench.basic_stats(res.time_us))
```

Smart enough AI could still probe random bits of memory

<https://github.com/gpu-mode/pygpubench>

Thank you many friends!

The KernelBot dev team: Matej Sirovatka, Erik Schultheis, Alex Zhang, Yang Wang, Jack Khuu, Ben Horowitz

Our sponsors: NVIDIA (esp the CuteDSL and cuda compute teams), AMD, Modal, Northflank

A big thank you to many Stanford researchers: Simon Guo, Mert Yuksekgonul, Natalia Kokoromyti

And the broader GPU MODE community

Get Involved



A Holistic Leaderboard

Shared goal: a filterable leaderboard where developers find the best agent/model for their scenario

computeeval.nvidia.com/leaderboard

Live Preview (Mock)

Architecture: B200 + H100

CUDA: 13.1

Domain: AI Kernels

Sort: Overall Score

Open-weight only: Off

Apply Filters

#	Model	Arch	CUDA	Domain	Correctness	Overall
1	Claude Opus 4.6 (thinking)	B200	13.1	AI kernels	93.0%	91.2
2	Gemini 3.1 Pro	H100	13.1	AI kernels	86.6%	87.4
3	GPT 5.2 high	H100	13.1	AI kernels	83.1%	84.6
4	KernelBot-SFT-v2 (open)	B200	13.1	AI kernels	76.4%	79.1
5	gpt-oss-120b-high	H100	13.1	AI kernels	68.5%	73.2

Scenario Match

Architecture

B200, H100

CUDA Toolkit

13.1 (target)

Recommended model

Claude Opus 4.6

Best correctness + strong performance under selected filters

Data source: ComputeEval + KernelBot (illustrative mock)

Last updated: Jan 2026

Architecture

Filter by GPU family and compute capability

CUDA Version

Target specific toolkit versions and features

Problem Domain

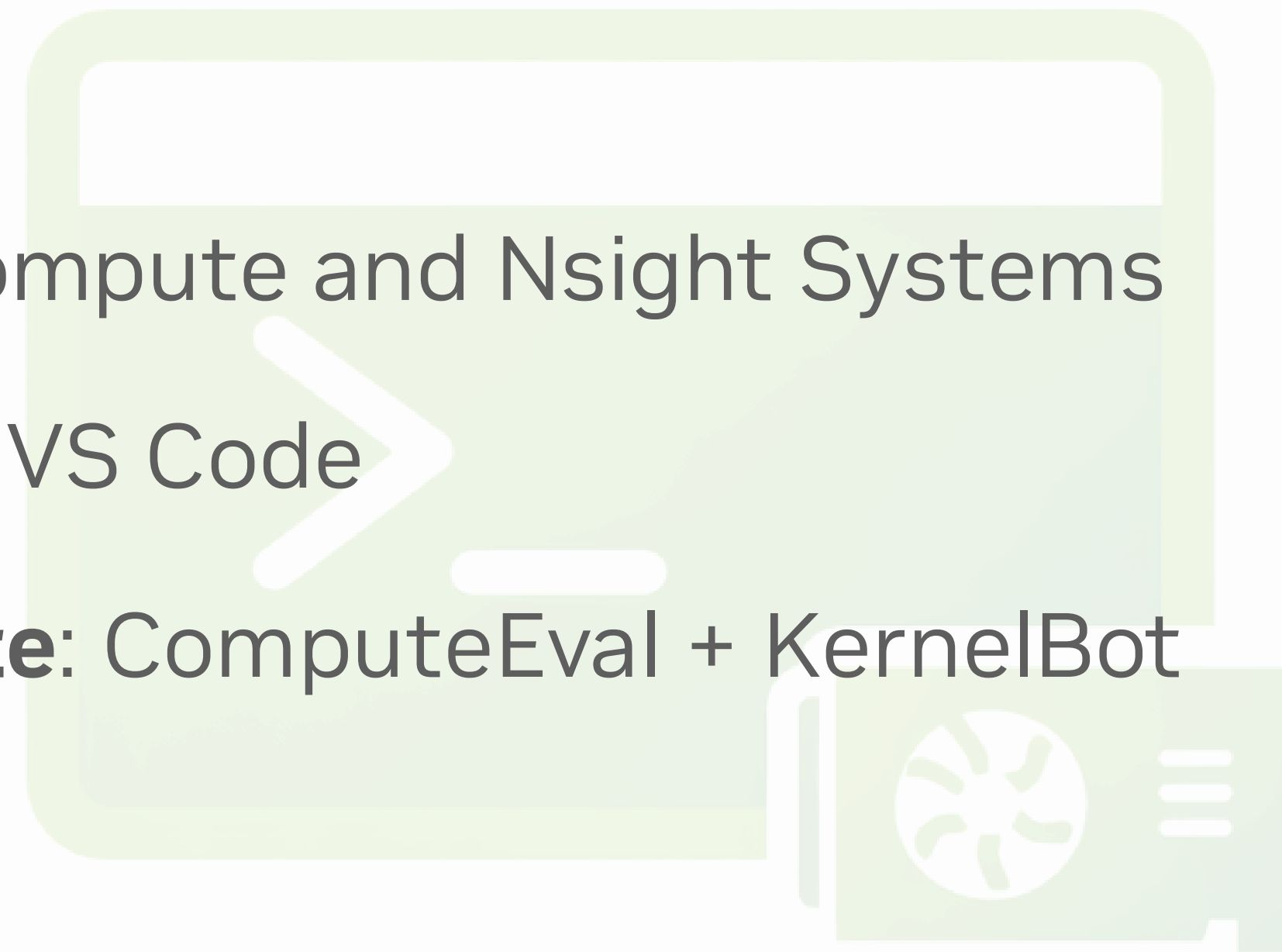
AI kernels, graphics, scientific computing, and more

For Developers and Researchers

Pick your lane and start this week

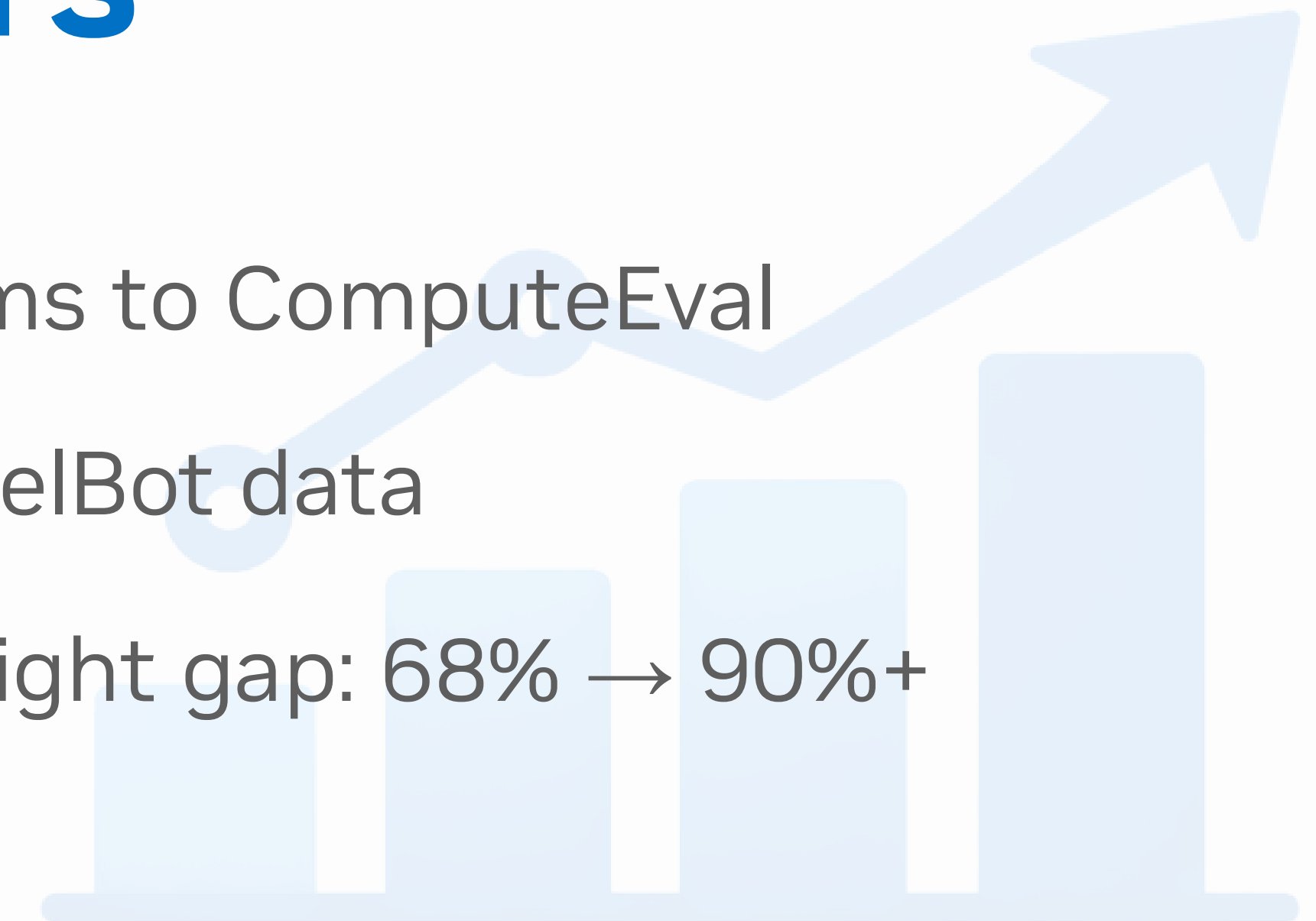
Developers

1. **Profile** with Nsight Compute and Nsight Systems
2. **Try** Nsight Copilot for VS Code
3. **Benchmark & compete:** ComputeEval + KernelBot



Researchers

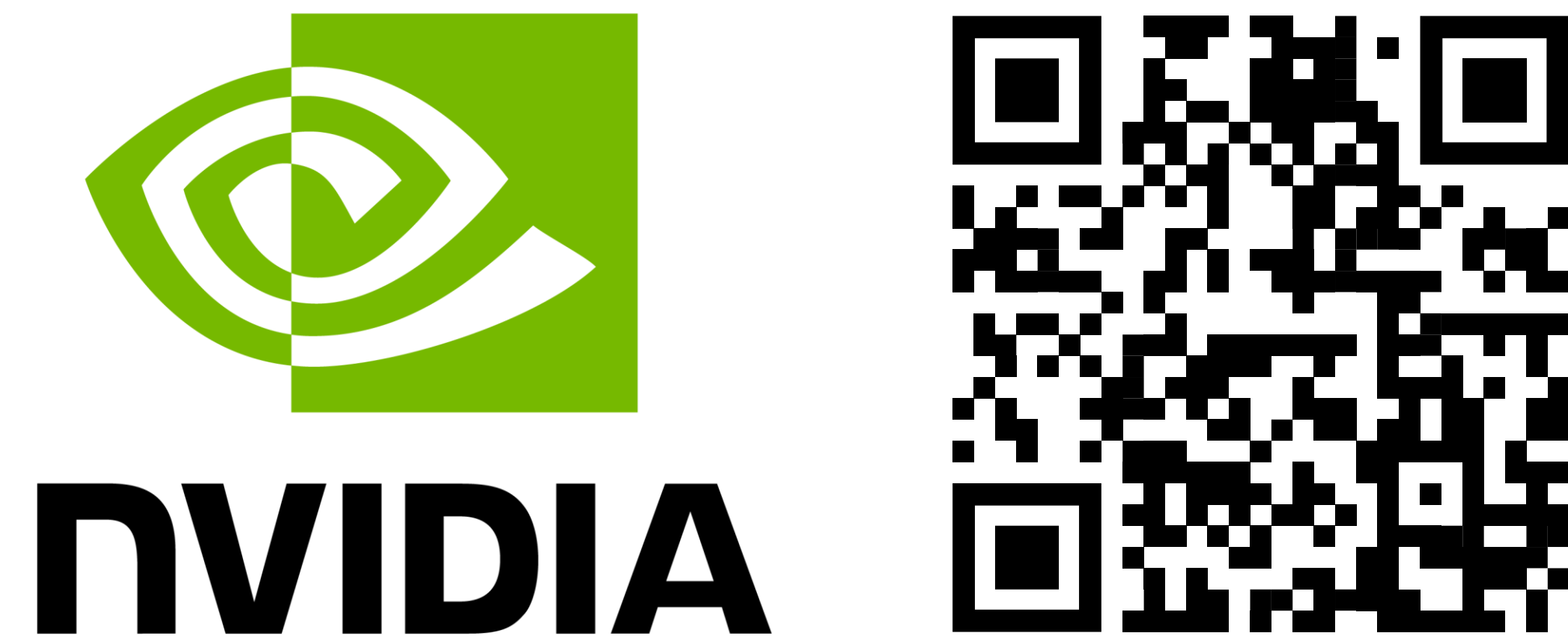
1. **Contribute** problems to ComputeEval
2. **Train** on open KernelBot data
3. **Close** the open-weight gap: 68% → 90%+



Shared goal: Close the correctness-performance gap

Community Resources

Two entry points for this talk's resources



ComputeEval — github.com/NVIDIA/compute-eval

Nsight Compute — developer.nvidia.com/nsight-compute

Nsight Systems — developer.nvidia.com/nsight-systems

Nsight Copilot — developer.nvidia.com/nsight-copilot

CUDA Toolkit — developer.nvidia.com/cuda-downloads



Main hub — gpumode.com

Discord — discord.gg/gpumode

YouTube — youtube.com/@GPUMODE

KernelBot — github.com/gpu-mode/kernelbot

Dataset — hf.co/datasets/GPUMODE/kernelbot-data

Don't miss the GPU Mode Awards at GTC.

Developer Tools Across GTC

Sessions

- [S81590](#): Learn how New AI Coding Agents and Tools Unlock GPU Performance for Everyone
- [S81831](#): AI Coding for the GPU: Building a Coding Agent to Help GPU developers write Speed of Light Code
- [S81653](#): LLM-Generated CUDA Kernels: Are We There Yet?
- [S81772](#): Don't leave Tensors on the Table: Programming and Optimizing Tensor Cores

Labs

- [DLIT81545](#): Profiling Python and AI workloads with Nsight Compute
- [DLIT81641](#): Find the Bottleneck – Optimize AI pipelines with Nsight Systems
- [DLIT81579](#): Optimizing PyTorch models for high-performance inference with Nsight Deep Learning Designer

Connect with the Experts

- [CWES81549](#): What's in Your Developer Toolbox? CUDA, AI, and Graphics Profiling, Optimization, and Debugging Tools
- [CWES81535](#): CUDA Developer Best Practices
- [CWES81771](#): How to Run and Optimize Your Workloads on the NVIDIA Grace and Vera CPUs
- [QA82372](#): What's in Your Developer Toolbox? CUDA, AI, and HPC, Optimization, and Debugging Tools

Live Demos

Come and visit the Developer Tools pod at the NVIDIA booth during exhibition floor hours!

Developer Tools are **free** and packaged in the latest version of the CUDA Toolkit

<https://developer.nvidia.com/cuda-downloads>

Support is available via:

<https://forums.developer.nvidia.com/c/developer-tools>

More information at:

<https://developer.nvidia.com/tools-overview>

